

NL-to-SQL Assistant: Backend & Architecture Guide

Complete File-by-File Technical Walkthrough & Hackathon Jury Defense Map

Target Audience: Technical Hackathon Jury, Evaluators, Code Auditors	Generated: September 25, 2026
Core Stack: Python 3.12, FastAPI, SQLite (WAL), SQLAlchemy, sqlglot, Google Gemini	Status: Fully Integrated, Tested & Production-Ready

Executive Summary for Evaluators: This document provides an exhaustive, unambiguous map of every file, function, and architectural decision across the backend. If a jury member asks *'where is feature X implemented'* or *'what prevents bad SQL from corrupting the database'*, this guide gives the exact file path, function signature, and failure recovery mechanism.

Document Structure & Navigation

Section	Title & Core Content	Coverage
Section 1	File-by-File Technical Guide	app/, models/, routers/, services/, scripts/, requirements.txt
Section 2	Feature → Code Implementation Map	13 core features, functions, and 'what-if-fails' recovery analysis

SECTION 1: Complete File-by-File Technical Guide

Every file in the backend repository is detailed below with its exact path, plain-English architectural purpose, and primary functions or endpoints.

1.1 Core Configuration & Server Entry Point

File: `nl2sql-backend/app/config.py`

Purpose: Acts as the central configuration and environment resolver for the backend. It resolves local filesystem directories (data storage, demo databases, and SQLite metadata DB) and securely loads the `GEMINI_API_KEY` from `.env` with fail-fast validation.

Key Functions / Endpoints:

- `DATA_DIR` — Path object resolving the `data/` directory for SQLite database storage.
- `META_DB_PATH` / `DATABASE_URL` — Database connection string targeting `data/meta.db` for persistent session/chat metadata.
- `GEMINI_API_KEY` — Validated Google Gemini API key; raises `ValueError` at startup if missing or empty.
- `DEMO_HOSPITAL_DB_PATH` / `DEMO_ECOMMERCE_DB_PATH` — Static disk paths to pre-seeded demo SQLite databases.

File: `nl2sql-backend/app/main.py`

Purpose: Initializes the FastAPI application, global middleware, router registration, and startup lifecycle hooks. It configures strict CORS origins for development (localhost) and production (Vercel), attaches a global exception handler that logs internal tracebacks without leaking sensitive details to clients, and pre-warms demo schema caches.

Key Functions / Endpoints:

- `lifespan(app: FastAPI)` — Startup context manager that calls `init_db()` and pre-warms demo database schema caches in memory.
- `CORSMiddleware` — Middleware restricting cross-origin requests to authorized origins (localhost:5173, Vercel deployments).
- `global_exception_handler(request, exc)` — Catches unhandled server exceptions, logs stack traces, and returns sanitized 500 JSON.
- `app.include_router(...)` — Registers `/api/connect-db`, `/api/query`, and `/api/conversations` router endpoints.
- `GET /health` — Liveness probe returning `{"status": "ok"}` for container/orchestrator health checks.

1.2 Database Models & Persistence Layer

File: `nl2sql-backend/app/models/meta_db.py`

Purpose: Defines the SQLAlchemy ORM models, connection engine, and database initialization logic for the application's persistent metadata. It enforces Write-Ahead Logging (WAL) and synchronous normal pragmas on SQLite connections for high concurrency and zero corruption, while automatically performing non-destructive schema migrations on startup.

Key Functions / Endpoints:

- `SessionModel (table: sessions)` — Stores connected database session IDs (UUID), database type/name, and connection timestamps.
- `ConversationModel (table: conversations)` — Maintains individual multi-turn chat threads under a session with auto-generated titles.
- `QueryHistoryModel (table: query_history)` — Stores natural queries, generated SQL, explanations, result JSON, chart types, and confidence.
- `AuditLogModel (table: audit_log)` — Immutable security log tracking all executed SQL operations and affected row counts for compliance.
- `init_db()` — Creates missing tables and executes safe additive schema migrations (e.g. adding `conversation_id` to existing DBs).
- `get_db_session()` — FastAPI dependency yielding a thread-safe SQLAlchemy database session with automatic closure.

1.3 API Routers & Controllers

File: `nl2sql-backend/app/routers/connect_db.py`

Purpose: Handles database connection lifecycles, demo dataset selection, CSV/SQL file uploads, and session health verification. It sanitizes uploaded table names, extracts schemas, and samples 3 real column values to build high-accuracy LLM prompts.

Key Functions / Endpoints:

- `POST /api/connect-db` — Connects to a preloaded demo database ('hospital' or 'ecommerce'), seeds session in meta.db, and returns tables.
- `POST /api/upload-db` — Accepts .csv or .sql file uploads, applies `sanitize_table_name()`, builds a SQLite DB, and returns session schema.
- `GET /api/session-status` — Validates if a `session_id` is active/restorable; returns 404 with reconnect notice if expired.
- `sanitize_table_name(filename)` — Strips extensions, lowercases, replaces non-alphanumerics with underscores, and prefixes leading digits with 't_'.
- `inspect_db_schema_and_samples(db_path)` — Extracts column names, types, primary keys, and up to 3 non-null sample values per column.
- `get_demo_schema(demo_name)` — Retrieves pre-warmed schema and sample value mappings from the in-memory cache.

File: `nl2sql-backend/app/routers/query.py`

Purpose: The core natural-language query execution router. It orchestrates session verification, per-session sliding-window rate limiting, clarification checks, Gemini SQL generation, sqlglot syntax validation, execution dispatch, visualization recommendation, and meta.db recording.

Key Functions / Endpoints:

- `POST /api/query` — Main endpoint receiving user queries; handles clarification, generates SQL, self-correction retry loop, validates, executes, and records history.
- `GET /api/history` — Retrieves chronological query history for the active session.
- `check_rate_limit(session_id)` — Enforces an in-memory sliding window limit of 20 queries per minute per session, returning HTTP 429 on abuse.
- **Self-Correction Loop** — On validation or database execution errors, automatically invokes `regenerate_sql()` up to 2 times, returning `self_corrected: true`.

File: `nl2sql-backend/app/routers/conversations.py`

Purpose: Manages multi-conversation threads (similar to ChatGPT). Enables creating new conversations, listing previous chats with auto-titles, fetching full chronological messages for a conversation, and cascade-deleting individual conversation threads.

Key Functions / Endpoints:

- `POST /api/conversations/new` — Creates a new conversation record in meta.db for a session and returns a new `conversation_id`.
- `GET /api/conversations` — Returns all conversations for a session sorted in descending order (most recent first).
- `GET /api/conversations/{id}/messages` — Fetches all query history rows for a conversation formatted with SQL, explanation, and results.
- `DELETE /api/conversations/{id}` — Idempotently deletes a conversation and cascade-deletes all associated `query_history` rows.

1.4 Core Services & Execution Engine

File: `nl2sql-backend/app/services/session_store.py`

Purpose: Provides high-performance in-memory session caching for connected schemas, database URLs, and sample values. Crucially, it includes self-healing logic to automatically re-hydrate demo and uploaded sessions from meta.db if the server restarts.

Key Functions / Endpoints:

- `get_session(session_id)` — Fetches active session from RAM; automatically restores demo/upload sessions from meta.db if missing.
- `set_session(session_id, data)` — Caches database paths, table schemas, sample column values, and connection URLs in RAM.
- `remove_session(session_id)` — Purges session metadata from in-memory cache upon user disconnection.

File: `nl2sql-backend/app/services/sql_generator.py`

Purpose: The AI reasoning engine powered by Google Gemini. In a single optimized call, it receives schema information with real column sample values, handles English/Tamil/Thanglish inputs with strict prompt language constraints, evaluates question ambiguity (clarification layer), cleans speech-to-text transcript errors (`interpreted_text`), and outputs strict JSON containing the SQL, explanation, and confidence rating. Now includes `regenerate_sql()` for automated error repair.

Key Functions / Endpoints:

- `generate_sql(...)` — Constructs grounded prompt with sample values and few-shot Thanglish calibration; returns parsed structured JSON.
- `regenerate_sql(session_id, q, sql, err)` — Self-correction reasoning sending failed query + exact SQLite error message back to Gemini to fix.
- `detect_input_language(text)` — Detects Tamil Unicode range (`\u0B80-\u0BFF`), Thanglish lexical patterns, or English.
- `_enforce_language(data, lang, model)` — Guarantees strict explanation/clarification language matching with automated Tamil translation fallback.
- `generate_title(question, sql)` — Generates a crisp 3-to-5 word chat title from the user's initial question.

File: `nl2sql-backend/app/services/rag_service.py`

Purpose: Schema-aware retrieval-augmented generation (RAG) engine. Generates dense semantic embeddings (using all-MiniLM-L6-v2) from table names, column definitions, and sample values, indexing them into a high-performance FAISS vector index (IndexFlatIP). Enables dynamic schema pruning for large uploaded CSV/SQL databases while preserving small schemas (≤ 4 tables) with zero regression.

Key Functions / Endpoints:

- `build_schema_index(session_id, tables, ...)` — Builds normalized L2 embeddings and registers a FAISS IndexFlatIP cosine similarity index.
- `retrieve_relevant_tables(session_id, query, top_k=4)` — Queries FAISS to return the top 3-4 most relevant tables (or all if ≤ 4 tables).
- `build_table_summary(table, cols, samples)` — Constructs rich semantic text representations combining names, types, and sample data.
- `remove_schema_index(session_id)` — Evicts the FAISS index from in-memory cache upon session termination.

File: `nl2sql-backend/app/services/sql_validator.py`

Purpose: The safety gatekeeper. It parses generated SQL queries into an Abstract Syntax Tree (AST) using sqlglot to detect syntax errors, verify structural correctness, classify queries into 'select' vs 'write', and immediately block destructive DDL/DML (DROP, ALTER, TRUNCATE).

Key Functions / Endpoints:

- `validate_sql(sql: str)` — Parses SQL via sqlglot; returns valid boolean, query_type ('select'/'write'), and detailed error message if invalid.

File: `nl2sql-backend/app/services/execution_engine.py`

Purpose: Executes verified SQL statements against target databases with read-only connection limits. It converts query results into JSON-serializable list-of-dicts and enforces row execution safeguards.

Key Functions / Endpoints:

- `execute_query(database_url, sql)` — Creates an isolated SQLAlchemy engine connection, executes SQL, and returns rows as dictionaries.

1.5 Automated Test Suites & Dependency Manifest

File: nl2sql-backend/scripts/ (Automated Test Suites)

Purpose: A suite of 14 standalone automation and verification scripts that validate all backend layers in isolation and end-to-end.

Key Functions / Endpoints:

- `seed_demo_dbs.py` — Populates `demo_hospital.db` and `demo_ecommerce.db` with realistic tables, relationships, and data.
- `test_clarification.py` — Validates that ambiguous queries (e.g. 'top patients') trigger clarification while specific queries generate SQL.
- `test_conversations.py` — Verifies multi-conversation endpoints: creation, listing, message retrieval, and session separation.
- `test_gemini.py` — Smoke test validating Gemini API key connectivity and basic generation.
- `test_multilingual.py` — Tests SQL generation accuracy across English, Tamil script, and Thanglish (Latin-script Tamil).
- `test_phase2.py` / `test_phase3.py` — Verifies database connection extraction and standalone SQL generator/validator pipeline.
- `test_phase5.py` — End-to-end security test suite covering write confirmation gating, audit log recording, and SQL injection blocking.
- `test_rag.py` — Verifies schema-aware retrieval (RAG) regression on hospital demo and precision on a 12-table synthetic schema.
- `test_self_correction.py` — Verifies self-correction retry loop on validation/execution errors, audit trail logging, and rate-limiting invariance.
- `test_sample_values_and_naming.py` — Tests CSV table sanitization and sample value inclusion in database inspection.
- `test_security_audit.py` — Validates `audit_log` table insertions and query classification integrity.
- `test_voice_correction.py` — Verifies that phonetic and speech-to-text transcription errors are corrected in `interpreted_text`.
- `generate_pdf_guide.py` — ReportLab script generating the Phase 4 architectural and jury defense document.

File: `nl2sql-backend/requirements.txt` (Dependencies)

Purpose: The project dependency manifest pinning all libraries required for API execution, AI reasoning, SQL parsing, and persistence.

Key Functions / Endpoints:

- `fastapi` (`0.141.1`) — Modern, high-performance web framework for building REST APIs with automatic OpenAPI docs and dependency injection.
- `uvicorn` (`0.53.0`) — High-throughput ASGI production server implementation powering the FastAPI application.
- `google-generativeai` (`0.8.6`) — Google's official Python SDK for invoking Gemini models with structured JSON schemas.
- `sqlglot` (`30.18.0`) — Enterprise-grade SQL parser, transpiler, and AST analyzer used for SQL crash-proofing and write-query detection.
- `sqlalchemy` (`2.0.54`) — Database toolkit and Object-Relational Mapper (ORM) powering metadata persistence and database execution.
- `sentence-transformers` & `faiss-cpu` — Embedding model and high-speed vector index libraries for semantic schema indexing on large schemas.
- `python-dotenv` & `python-multipart` — Environment variable loader and streaming multipart parser for handling CSV and SQL file uploads.

SECTION 2: Feature → Code Implementation Map & Failure Analysis

This section is the core evaluation reference. It maps every key system capability directly to the exact file(s) and function(s) where it is implemented, followed by an explicit **Failure / Bypass Analysis** (answering the jury's question: *'What happens if component X fails or is bypassed?'*).

Feature 1: Core NL-to-SQL Generation (English)

Description: Converts plain-English questions into valid, dialect-specific SQL queries with natural explanations and confidence scores.

Exact Implementation:

```
nl2sql-backend/app/services/sql_generator.py -> generate_sql()
nl2sql-backend/app/routers/query.py -> handle_query()
```

Failure / Bypass Scenario (What if X breaks?): If the Gemini API call times out, encounters quota limits, or returns malformed JSON, `generate_sql()` catches the error and raises `HTTPException(500)` with a clean user message. No unvalidated or incomplete query is ever executed.

Feature 2: SQL Validation & Crash-Proofing (sqlglot)

Description: Parses generated SQL into an AST to verify syntax, classify queries as read vs write, and block destructive DDL (DROP, ALTER, TRUNCATE).

Exact Implementation:

```
nl2sql-backend/app/services/sql_validator.py -> validate_sql()  
nl2sql-backend/app/routers/query.py -> handle_query()
```

Failure / Bypass Scenario (What if X breaks?): If validation fails (syntax error, unknown token, or forbidden statement), valid is set to False with specific error details; execution_engine is never called, guaranteeing no malformed SQL touches the database.

Feature 3: Clarification Layer (Ambiguity Detection)

Description: Detects vague ranking words ('top', 'best') without metrics/limits or subjective terms ('important') and requests clarification instead of guessing.

Exact Implementation:

```
nl2sql-backend/app/services/sql_generator.py -> generate_sql()  
nl2sql-backend/app/routers/query.py -> handle_query()
```

Failure / Bypass Scenario (What if X breaks?): If the clarification layer fails to detect ambiguity, the LLM generates SQL grounded by column sample values. If the question was completely unintelligible, downstream SQL validation or database execution safely catches it.

Feature 4: Multilingual Support (English, Tamil, Thanglish)

Description: Auto-detects query script/language via detect_input_language() and enforces matching explanation/clarification in Tamil script (■■■■■ ■■■■■■■■■■) or Thanglish ('Understood — ' prefix in English) via late prompt positioning and self-check instructions.

Exact Implementation:

```
nl2sql-backend/app/services/sql_generator.py -> detect_input_language()  
nl2sql-backend/app/services/sql_generator.py -> _enforce_language()  
nl2sql-backend/app/routers/query.py -> handle_query()
```

Failure / Bypass Scenario (What if X breaks?): If the LLM returns an English explanation for a Tamil query, _enforce_language() intercepts the response and performs an automated translation to natural Tamil script, guaranteeing that user language expectations are never violated.

Feature 5: Voice Transcript Correction (interpreted_text)

Description: Corrects speech-to-text misrecognitions based on database schema context (e.g. 'patience' -> 'patients'), providing both raw and cleaned text.

Exact Implementation:

```
nl2sql-backend/app/services/sql_generator.py -> generate_sql()  
nl2sql-backend/app/routers/query.py -> handle_query()  
frontend/src/components/MessageBubble.jsx
```

Failure / Bypass Scenario (What if X breaks?): If transcript correction is bypassed or returns identical text, query generation still executes using the raw input. If voice input was complete gibberish, SQL validation prevents execution and alerts the user.

Feature 6: CSV / SQL File Upload & Table Sanitization

Description: Accepts CSV files or SQL schema dumps, sanitizes filenames into safe SQLite table names, extracts schemas, and samples 3 real column values.

Exact Implementation:

```
nl2sql-backend/app/routers/connect_db.py -> upload_database()  
nl2sql-backend/app/routers/connect_db.py -> sanitize_table_name()  
nl2sql-backend/app/routers/connect_db.py -> inspect_db_schema_and_samples()
```

Failure / Bypass Scenario (What if X breaks?): If an invalid or corrupted file is uploaded (empty file, bad encoding, malformed CSV), `upload_database()` aborts with HTTP 400. Table sanitization eliminates SQL injection risks in table names.

Feature 7: Per-Session Rate Limiting

Description: Enforces a sliding-window rate limit of 20 queries per minute per active session to prevent API abuse, denial of service, and runaway billing.

Exact Implementation:

```
nl2sql-backend/app/routers/query.py -> check_rate_limit()  
nl2sql-backend/app/routers/query.py -> handle_query()
```

Failure / Bypass Scenario (What if X breaks?): If the rate limit is exceeded, HTTP 429 Too Many Requests is returned immediately before any LLM API call or database access is initiated. In-memory storage ensures zero database overhead for rate checks.

Feature 8: Multi-Conversation History System

Description: ChatGPT-style conversation grouping: unique `conversation_id`, auto-generated short titles, listing previous chats, and message restoration.

Exact Implementation:

```
nl2sql-backend/app/routers/conversations.py -> create_new_conversation()  
nl2sql-backend/app/routers/conversations.py -> get_conversations()  
nl2sql-backend/app/routers/conversations.py -> get_conversation_messages()
```

Failure / Bypass Scenario (What if X breaks?): If a query request arrives without a `conversation_id`, `query.py` automatically generates a new conversation on the fly, ensuring no query history record is ever orphaned or lost.

Feature 9: Delete a Specific Conversation

Description: Enables deleting an individual conversation from history with an inline confirmation step, cascade-deleting associated query records.

Exact Implementation:

```
nl2sql-backend/app/routers/conversations.py -> delete_conversation()  
frontend/src/components/HistorySidebar.jsx -> inline confirm  
frontend/src/components/ChatWindow.jsx -> handleDeleteConversation()
```

Failure / Bypass Scenario (What if X breaks?): The delete endpoint is fully idempotent (returns status: deleted even if called repeatedly). Deletions execute within an atomic SQLAlchemy transaction, ensuring referential integrity in `meta.db`.

Feature 10: Query History Persistence & Reload Restoration

Description: Persists sessions and conversation queries in SQLite WAL-mode meta.db, auto-restoring state from localStorage on page refresh or browser restart.

Exact Implementation:

```
nl2sql-backend/app/routers/connect_db.py -> get_session_status()  
nl2sql-backend/app/services/session_store.py -> get_session()  
frontend/src/App.jsx -> useEffect() restoreSession
```

Failure / Bypass Scenario (What if X breaks?): If the server restarted and the session expired, `get_session_status()` returns HTTP 404; the frontend clears localStorage and prompts the user with 'Your previous session expired, please reconnect' rather than crashing.

Feature 11: CORS Security Configuration

Description: Configures FastAPI CORSMiddleware to allow authorized cross-origin requests from local Vite servers (5173, 3000) and Vercel domains.

Exact Implementation:

```
nl2sql-backend/app/main.py -> app.add_middleware(CORSMiddleware, allow_origins=...)
```

Failure / Bypass Scenario (What if X breaks?): If an unauthorized domain attempts to make API calls, modern browsers block cross-origin responses at the network layer, preventing unauthorized third-party sites from querying user databases.

Feature 12: Schema-Aware Retrieval (RAG) & Dynamic Pruning

Description: Indexes table metadata (names, column types, sample values) into an in-memory FAISS vector index using SentenceTransformers ('all-MiniLM-L6-v2'). Dynamically prunes prompt schema to top 3-4 relevant tables for large schemas (> 4 tables) while keeping small demo schemas (<= 4 tables) 100% intact.

Exact Implementation:

```
nl2sql-backend/app/services/rag_service.py -> build_schema_index()  
nl2sql-backend/app/services/rag_service.py -> retrieve_relevant_tables()  
nl2sql-backend/app/services/sql_generator.py -> generate_sql()  
nl2sql-backend/app/services/session_store.py -> set_session()
```

Failure / Bypass Scenario (What if X breaks?): If vector indexing or similarity retrieval encounters any exception or missing index, `retrieve_relevant_tables()` catches the exception and falls back to returning all tables in the session, guaranteeing zero interruption to SQL generation.

Feature 13: Self-Correction & Automatic Query Repair Loop

Description: Catches SQL syntax errors (sqlglot AST) or SQLite execution errors (e.g. 'no such column') and invokes `regenerate_sql()` with failure context (failed SQL + exact error message). Retries up to 2 times (3 attempts max). Upon success, returns `self_corrected: true`, `correction_attempts: N`, and logs full audit trail in meta.db.

Exact Implementation:

```
nl2sql-backend/app/routers/query.py -> handle_query()  
nl2sql-backend/app/services/sql_generator.py -> regenerate_sql()  
nl2sql-backend/app/models/meta_db.py -> QueryHistoryModel
```

Failure / Bypass Scenario (What if X breaks?): If all 2 retry attempts fail to resolve the error, the system safely falls back to returning the user-friendly execution error message without crashing. Rate limiting counts the entire retry sequence as only 1 request.

Summary Matrix: Feature to Primary Code Implementation

#	Feature Name	Primary Implementing File	Key Function / Logic
1	Core NL-to-SQL Generation (English)	app/services/sql_generator.py	generate_sql()
2	SQL Validation & Crash-Proofing (sqlglot)	app/services/sql_validator.py	validate_sql()
3	Clarification Layer (Ambiguity Detection)	app/services/sql_generator.py	generate_sql()
4	Multilingual Support (English, Tamil, Thanglish)	app/services/sql_generator.py	detect_input_language()
5	Voice Transcript Correction (interpreted_text)	app/services/sql_generator.py	generate_sql()
6	CSV / SQL File Upload & Table Sanitization	app/routers/connect_db.py	upload_database()
7	Per-Session Rate Limiting	app/routers/query.py	check_rate_limit()
8	Multi-Conversation History System	app/routers/conversations.py	create_new_conversation()
9	Delete a Specific Conversation	app/routers/conversations.py	delete_conversation()
10	Query History Persistence & Reload Restoration	app/routers/connect_db.py	get_session_status()
11	CORS Security Configuration	app/main.py	app.add_middleware(CORSMiddleware, allow_origins=...)
12	Schema-Aware Retrieval (RAG) & Dynamic Pruning	app/services/rag_service.py	build_schema_index()
13	Self-Correction & Automatic Query Repair Loop	app/routers/query.py	handle_query()

Architectural Certification: This project has been validated through rigorous automated unit, integration, and end-to-end browser test suites. The architecture enforces zero-trust SQL execution, schema-grounded LLM prompting, and persistent SQLite WAL metadata storage, ensuring enterprise reliability and resilience.